

ManicBot

Мультиотенантная платформа записи в салоны красоты:
Cloudflare Worker, D1, KV, Telegram и дополнительные каналы.
Полный обзор структуры кода, потоков данных и админского
Mini App.

Кратко. Проект состоит из двух развёртываемых частей: **Worker** (`manicbot/`) обрабатывает вебхуки, биллинг, cron и прокси лендинга; **Admin Mini App** (`manicbot/admin-app/`) — Next.js 15 + tRPC + Drizzle для ролевых дашбордов в Telegram WebApp. Общая БД — D1; эфемерное состояние и токены — KV. Автотесты Worker: **825** тестов в **47** файлах (Vitest).

Содержание

1. Карта репозитория
2. Слои и границы системы
3. Worker: маршрутизация и контекст
4. Омниканальность (Telegram, WhatsApp, Instagram)
5. Данные: D1, KV, миграции
6. Доменная логика в Worker
7. Admin Mini App
8. Безопасность и роли
9. Качество и тесты
10. Выводы и рекомендации

1. Карта репозитория

Корень рабочей копии содержит в основном каталог `manicbot/`. Исходный код Worker сосредоточен в `manicbot/src/` (~100+ модулей JS плюс i18n для en/pl/ru/ua). Тесты — `manicbot/test/*.test.js`. Схема SQL и миграции — `src/db/schema.sql`, `migrations/`. Конфигурация Cloudflare — `wrangler.toml` (маршруты `manicbot.com`, привязки KV `MANICBOT`, D1 `DB`, AI binding, cron каждые 15 минут).

Путь	Назначение
<code>manicbot/src/worker.js</code>	Единая точка входа HTTP + scheduled handler
<code>manicbot/src/handlers/</code>	Сообщения, колбэки, cron, унифицированный inbound
<code>manicbot/src/tenant/</code>	Резолвинг бота → tenant, D1 registry, миграции KV→D1
<code>manicbot/src/channels/</code>	Адаптеры Meta (WA/IG), Telegram bridge, UI renderer
<code>manicbot/src/services/</code>	Записи, пользователи, чат, календарь, тикеты
<code>manicbot/src/billing/</code>	Stripe, webhooks, фичи по плану
<code>manicbot/src/ui/</code>	Экраны, клавиатуры, сценарии бронирования
<code>manicbot/admin-app/src/</code>	Next App Router, tRPC routers, компоненты дашбордов

2. Слои и границы системы

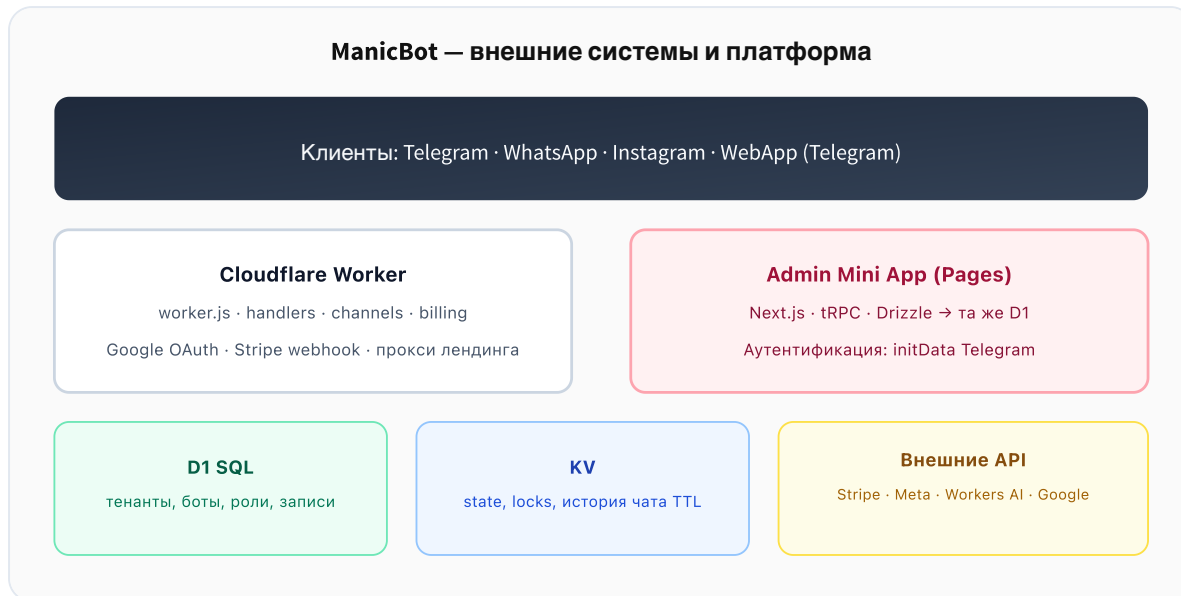


Рис. 1. Два приложения (Worker и Pages) делят D1; Worker интегрирует каналы и вебхуки.

3. Worker: маршрутизация и контекст

Функция `fetch` в `worker.js` обрабатывает цепочку путей: лендинг (прокси на `LANDING_URL`), Stripe (`/stripe/webhook`, success page), админские эндпоинты с ключом (`/admin/migrate`, `seed`, `provision`), Google Calendar OAuth, HTML-админка `/admin` и выгрузки CSV, публичные ICS по `/calendar/:id`, Telegram webhook `/webhook/:botId` (и legacy `/webhook`), а также `/webhook/wa` и `/webhook/ig` для Meta.

Контекст запроса. Для путей вида `POST /webhook/{botId}` вызывается `resolveTenantFromBotId` и `buildTenantCtx`. Если миграция на D1 ещё не завершена или бот не найден, используется `buildLegacyCtx` / `buildCtx` — это сохраняет обратную совместимость с одноботовой конфигурацией через секреты `BOT_TOKEN`.

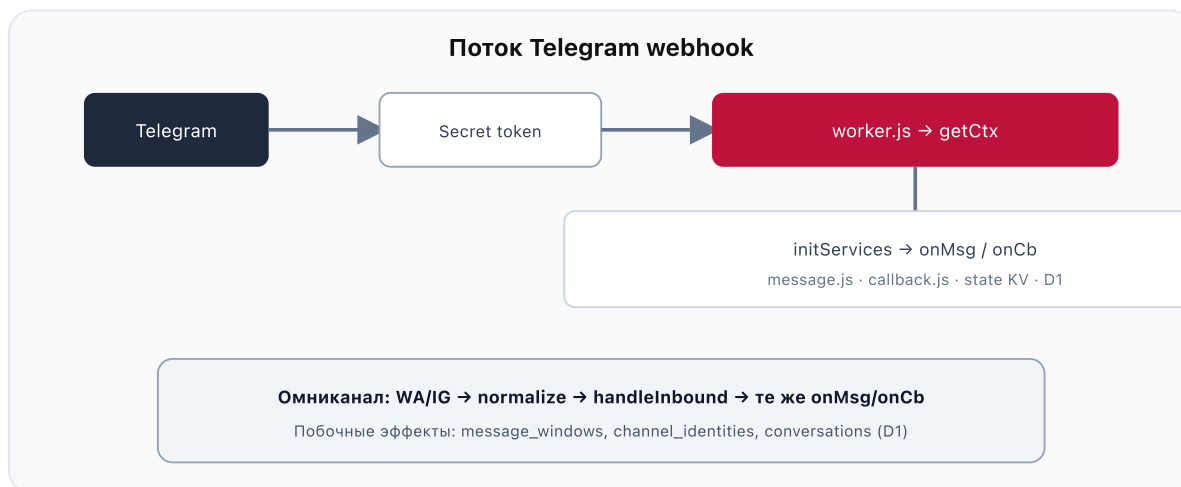


Рис. 2. Единый доменный обработчик сообщений после нормализации канала.

4. Омниканальность

Модуль `handlers/inbound.js` принимает нормализованный `InboundMessage`, обновляет окна сообщений для Meta-каналов, идентичности и строки «inbox» в D1, затем вызывает существующие `onMsg` / `onCb` с псевдо-объектами Telegram — это уменьшает дублирование UI-логики. Проверка подписи Meta — `channels/meta-verify.js`; резолвинг тенанта по `phone_number_id` / `page_id` — `channels/resolver.js`.

5. Данные

D1 хранит мультитенантный реестр (`tenants`, `bots`), роли (`tenant_roles`, `platform_roles`), бизнес-сущности (записи, мастера, услуги, пользователи), биллинговые поля тенанта, тикеты, а также таблицы для каналов и диалогов (в т.ч. `conversations`, `message_windows`).

KV используется для состояния диалога, блокировок, зашифрованных токенов ботов и короткоживущей истории для AI (см. `utils/kv.js`). Префиксы документированы в `CLAUDE.md` (`t:{tenantId}`, `state:{cid}` и т.д.).

6. Доменная логика Worker

- **Запись и расписание:** `services/appointments.js`, UI в `ui/booking.js`, `ui/screens.js`.
- **Пользователи и роли:** `services/users.js`, `roles/roles.js`; God Mode через `ADMIN_CHAT_ID`.
- **AI:** `ai.js` — Workers AI с fallback моделей и лимитами; теги действий в ответах.
- **Биллинг:** Stripe webhooks обновляют статус в D1; `billing/features.js` ограничивает возможности по плану.
- **Уведомления и календарь:** `notifications.js`, Google Calendar OAuth в `services/google-calendar-oauth.js`, экспорт ICS.
- **i18n:** каталоги `i18n/en|pl|ru|ua` для текстов бота.

7. Admin Mini App

Стек: Next.js 15, tRPC 11, SuperJSON, Drizzle, Tailwind 4. Точка API: `app/api/trpc/[trpc]/route.ts`. Роутеры объединены в `server/api/root.ts`: `auth`, `salon`, `master`, `support`, `channels`, `conversations`, плюс админские `system`, `metrics`, `users`, `billing`, `tenants`, `appointments`, `export`, `provisioning`, `settings`.

Аутентификация: заголовок `x-telegram-init-data` → HMAC-проверка в `server/auth/telegram.ts`. Контекст tRPC создаётся в `trpc.ts`. Для владельцев салона доступ к tenant-scoped процедурам централизован (например `tenantAccess.ts` — проверка `tenant_owner` или `system admin`).

UI: `TelegramGate.tsx` маршрутизирует по ролям; дашборды `SalonDashboard` , `MasterDashboard` , `SupportDashboard` ; God Mode — страницы в `app/*` .

8. Безопасность и роли

Платформенные роли: `system_admin` (в т.ч. по `ADMIN_CHAT_ID`), `support` , `technical_support` ; арендные: `tenant_owner` , `master` ; клиент по умолчанию без Mini App.

Замечание по коду. В `adminProcedure` (`trpc.ts`) явно разрешены роли из `platform_roles` : `system_admin` и `support` . Если для части админских страниц требуется `technical_support` , стоит убедиться, что соответствующие процедуры это учитывают (отдельные роутеры `support` могут иметь свою матрицу).

Worker защищает вебхуки секретом Telegram, Meta — HMAC; админские HTTP-пути — `ADMIN_KEY` или Basic там, где вызывается `checkAdmin` .

9. Качество и тесты

Покрываем широко по вертикали: резолвер арендатора, биллинг, каналы, AI-колбэки, i18n-целостность, безопасность, клавиатуры, KV, даты, ICS, лендинг-прокси, worker без хардкода токенов и др. Это снижает регрессии при изменении маршрутизации и биллинга.

В `admin-app` есть точечные тесты (`src/__tests__`) — основной объем автоматизации сосредоточен в Worker.

10. Выводы и рекомендации

Сильные стороны. Чёткое разделение Worker / Mini App; мультитенантность через D1; единый inbound для омниканала; документированная модель ролей в `CLAUDE.md`; зрелый набор тестов на Worker; интеграции Stripe, Google Calendar, Workers AI собраны в предсказуемых модулях.

Риски и долг. Сосуществование legacy и D1-путей усложняет ментальную модель — важно постепенно сужать legacy. `worker.js` очень большой: при росте фич имеет смысл выносить группы маршрутов в отдельные модули. Дублирование схемы D1 между Worker SQL и Drizzle-схемой требует дисциплины при миграциях.

Практические шаги. (1) Периодически прогонять `npx vitest run` перед деплоем. (2) Держать секреты только в Cloudflare, не в репозитории. (3) При добавлении каналов расширять `InboundMessage` и тесты в `test/channels-*.test.js`. (4) Для Mini App — выровнять матрицу ролей для всех админских процедур с продуктовыми требованиями.

Конец отчёта · Исходник HTML: docs/ManicBot_Architecture_Review.html